Modeling, Pricing and Applications of Parisian Options

Souleymane OUATTARA
Aboubakar OUATTARA
Christ Ange Dylan KOUAME

Referring teacher : Anne EYRAUD

April 14, 2025

Contents

List of Figures

Abstract

Barrier options are exotic financial instruments whose activation or cancellation depends on crossing a predefined level. Among them, Parisian options stand out due to an additional monitoring mechanism that tracks the time spent above or below the barrier. This feature makes their modeling and pricing more complex.

In this work, we provide an in-depth study of Parisian options by exploring several pricing approaches, including the Black-Scholes model, the binomial tree method, and Monte Carlo simulation. We analyze the impact of the monitoring mechanism on valuation and discuss practical applications of these options in finance. Finally, we compare different numerical methods to assess their efficiency and limitations in a realistic framework.

The results highlight the importance of selecting the appropriate model based on market characteristics and investor requirements.

Introduction

Financial options play a central role in risk management and derivative structuring. Among them, *barrier options* introduce a dependency on a predefined price level, making their valuation more complex than standard European or American options. Within this category, *Parisian options* add an additional dimension: their activation or deactivation depends not only on crossing a barrier but also on the time spent above or below it. This monitoring mechanism introduces mathematical and comPutational challenges that require specific modeling approaches.

In this work, we present a comprehensive study of Parisian options by exploring different valuation methods and their implications. The first part introduces the fundamental concepts of options, focusing on barrier options and their Parisian variants. The second part is dedicated to the binomial tree method and its adaptation to the Parisian framework. The third part explores the Black-Scholes model as well as the use of Monte Carlo simulation for pricing these options. The fourth part provides a comparative analysis of the different methods, highlighting their advantages and limitations based on market parameters and option characteristics. Finally, the fifth part extends the analysis to practical applications of Parisian options, not only in finance but also in areas such as deposit guarantee schemes and bankruptcy processes in the United States.

This work aims to provide a global and comparative perspective on the pricing methods of Parisian options while emphasizing their relevance in various economic and industrial contexts.

1 Fundamentals of Financial Options and Parisian Options

1.1 General Case of European Options

An option is a right to buy or sell a given asset within a defined period. It can be either a Put or a Call. A Put grants the right to sell, while a Call grants the right to buy. Various types of options exist in financial markets, including European, American, and Asian options, each with specific characteristics and different levels of flexibility.

European options allow buying (Call) or selling (Put) only at expiration. They are less common in stock markets and are cheaper than American and Asian options due to their limited flexibility. The option's value consists of its intrinsic value (the difference between the underlying asset's price and the strike price) and its time value. The time value decreases as the option approaches expiration.

1.2 Barrier Options

They belong to the family of *path-dependent options*, meaning their value depends on the trajectory of the underlying asset.

Like a vanilla European option, the payoff of a barrier option, sometimes called a *limit option*, depends on the difference between the underlying asset's price at maturity and the predetermined strike price. However, this type of contract includes a condition linking its exercise to the underlying asset crossing (or failing to cross) a specified threshold (the **barrier**).

There are two main types of barrier options: **knock-in** and **knock-out**. A barrier option is either activated or deactivated when the underlying asset reaches a predetermined upper or lower barrier. This results in eight types of barrier options, classified according to three criteria:

- **Call or Put** (buy or sell option)
- **Knock-in or Knock-out** (activated or deactivated)
- **Up or Down** (barrier crossed from below or above)

1.3 Focus on Parisian Options

Parisian options were **introduced in 1997 by Chesney, Jeanblanc, and Yor**. These options are extensions of barrier options, structured to be more secure for the buyer. Similar to barrier options, there are eight distinct types of Parisian options (e.g., Parisian up-and-out Call).

What distinguishes Parisian options from barrier options? A major drawback for a buyer of a simple knock-out barrier option and for a seller of a simple knock-in barrier option is that the option is lost (or created) even if the underlying asset touches the barrier only once and then permanently moves away from it.

To address this issue and **prevent potential price manipulations**, a Parisian option is activated or deactivated only if the **underlying asset remains strictly** (i.e., without crossing back) beyond the barrier for a minimum duration.

Thus, a Parisian *Call up-and-in* with characteristic parameters L and D is activated only if the underlying asset remains above level L for more than duration D after reaching it, without crossing back, and before maturity.

For example:

- **Parisian Knock-in**: The option becomes active if the underlying asset stays above/below the barrier for a given consecutive period.
- **Parisian Knock-out**: The option is canceled if the underlying asset remains beyond the barrier for a certain consecutive period.

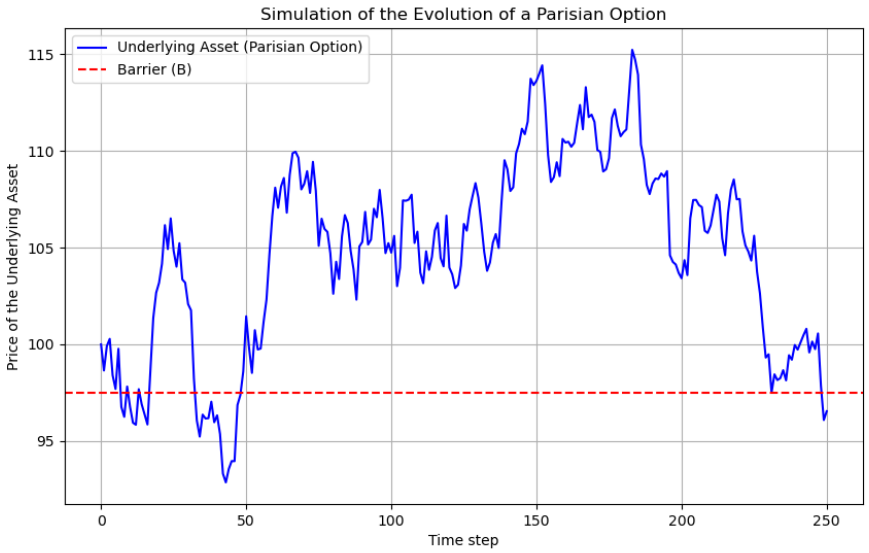


Figure 1: Down-Type Barrier

2 Modeling and Valuation of Barrier Options

As mentioned in the introduction, there are two families of barrier options known as 'Knock-out' and 'Knock-in'. In the first family, the option disappears as soon as the price of the underlying asset touches the barrier. Whereas, in the second family, the option comes into existence as soon as the price of the underlying asset touches the barrier.
Moreover, it is important to note that:

- A 'knock-in' option can be valued from a 'knock-out' option and vice versa.
- The combination of a 'Down-and-out' option and a 'Down-and-in' option forms a standard call option.
- It is easy to determine the price of a 'Down-and-in' by subtracting the price of a 'Down-and-out' from the price of a standard call option.
- The combination of an 'Up-and-out' option and an 'Up-and-in' option forms a standard call option.

Thus, in all the valuation models that will follow, we will consider the **case of 'knock-in' options** and an asset that does **not pay dividends**, both for **barrier options** in general and for **Parisian options in particular.**
Furthermore, it is often the case that the contract includes a clause stating that, in the event of non-activation of the "In" option or deactivation of the "Out" option, the buyer is entitled to a **Rebate** (discount, consolation prize) in cash, fixed in advance. Naturally, options with a **rebate** are more expensive to purchase than their counterparts without one.
In our case, we will model and evaluate barrier options that do not include a **rebate**.

2.1 Binomial Model

2.1.1 General Framework: Valuation of Vanilla Options

The idea of the method is to discretize time into several observation periods, and to construct a tree in which the price of the option can either go up or down from one step to the next. The option is then valued by backward induction through the tree from the option values at maturity. The hypotheses are as follows:

- Discretization of time. We divide the time to maturity of the option (T) into N periods of length $\Delta t = \frac{T}{N}$.
- The price of the underlying asset evolves according to a binomial process. The price of the underlying can either go up by a factor of "u" or go down by a factor of "d".
- The risk-free interest rate (r) is constant. It will be used for discounting future cash flows and calculating the risk-neutral probability. R is the discount factor .
- The market operates in no-arbitrage condition, which is equivalent to the following inequality: $u < R < d$
- There are no transaction costs or dividends.

- The investor can sell short and borrow at the risk-free rate. This hypothesis allows us to replicate the option's payoff using a combination of the underlying asset and borrowing at the risk-free rate. This is referred to as a **simple portfolio strategy**.

First, we collect our various variables:

- S_0: the initial price of the underlying asset
- T: the maturity of the option
- N: the number of periods
- σ: the volatility of the underlying asset
- $u = e^{\sigma\sqrt{\Delta t}}$: the up factor
- $d = e^{-\sigma\sqrt{\Delta t}}$: the down factor
- V_t^u and V_t^d: the different states of the option at time t
- $C_T = max(S_T - K, 0)$: the payoff of a Call option

The calibration of the up and down parameters was done according to the COX-ROSS-RUBINSTEIN model.

The idea of this model is to find a risk-neutral probability $\mathbb{Q}$ that makes every self-financing simple portfolio a martingale, thus allowing us to deduce the option's value as the discounted expectation under $\mathbb{Q}$ of the option's payoff.

For the calculation of the risk-neutral probability $\mathbb{Q}$, we must first ensure its existence, which directly follows from the hypothesis of no arbitrage opportunities, specifically from the relation that this implies $(u < R < d)$.

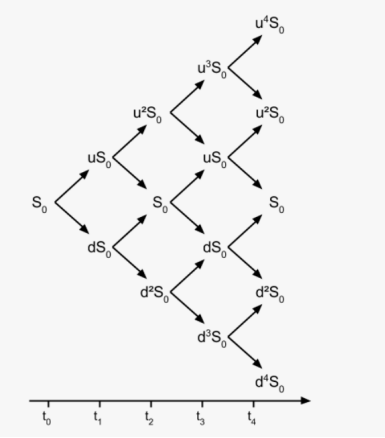


Figure 2: Example of binomial tree

To backtrack through the tree, we therefore use the following relation:

$$V_t = \frac{1}{R}\mathbb{E}^{\mathbb{Q}}(V_{t+1}) \tag{1}$$

$$= \frac{1}{R} \times (qV_{t+1}^u + (1-q)V_{t+1}^d) \tag{2}$$

which gives, by recurrence:

$$V_0 = \frac{1}{\mathrm{R}^n} \sum_{t=0}^{N} \binom{n}{k} q^k (1-q)^{n-k} max\{u^i d^{n-i} S_T - K, 0\}$$

Where $\mathbb{Q} = \frac{R-d}{u-d}$

2.1.1.1 Case of Barrier Options

In the context of barrier options, the reasoning will be almost identical, with one key difference: the barrier.

Indeed, in the case of "knock-in" options, as long as the underlying asset does not reach the barrier, the option does not exist. Once it is reached, the option is activated and then behaves like a vanilla option, whose value we know how to assess.

Thus, by denoting the barrier as B, we need to check at each period whether "$S_t \geq B$" (for an "up" barrier) or "$S_t \leq B$" (for a "down" barrier). Then, we determine the final payoffs of the paths stemming from this node once the condition is met, and proceed by backward induction using relation (1).

2.1.1.2 Specificity of Parisian Options

The modeling of Parisian options is similar to that of barrier options discussed earlier, but with one major difference: **the minimum duration**.
Thus, an additional variable is introduced, alongside those already defined for standard European and barrier options.

- G: a random varibale which tells if the option spent at least D steps over the barrier or not. If it takes the value 0, it means that we haven't spent continuously the number of steps required

We will define it for an up-and-in call option as:

$$G = \sum_{k=0}^{T-1-D} \prod_{i=k}^{D+k} \mathbb{1}_{S_i \geq B}$$

Where D represents the time required to activate or disable the option.

In this expression, we consider the set of paths of length d that remain above the barrier, where D represents the minimum time required for the barrier to be activated. Thus, the option is triggered as soon as the variable D differs from zero.

So the terminal value of the options becomes:

$$V_T = (S_T - K)_+ \mathbb{1}_{D \geq 1}$$

And after determining the **final payoffs** of the paths stemming from the corresponding terminal node, we then proceed as before with **backward induction**, using the relation (1).

2.1.2 Numerical Implementation

Regarding the implementation of the tree, we chose to start with the pricing of an up-and-in Call.

We first initialize the variables.

Next, we compute the different values that will be used later and store them in variables to simplify notation. Then, we create three tables: the first to store the values of the underlying asset, the second to store those of the option, and the third to indicate whether a path is above or below the barrier at first, though it will serve a broader purpose later on. Finally, we can start the algorithmic process.

STEP 1: Filling the table containing the values of the underlying asset.

STEP 2: Generating all possible paths in an $N \times N$ matrix using the itertools library and creating a table called *activate*, with a size equal to the number of possible paths, containing Boolean values initially set to False.

STEP 3: Iterating through all paths using a loop, where each path starts at coordinate (1,1) in the considered matrix. For each path, we create a vector to store the coordinates of the different points the path passes through, and we initialize a counter to track the time spent above the barrier in the corresponding path.

We begin at coordinate (1,1) and move downward in the matrix at each step. We check if the underlying asset is above the barrier—if so, we increment the counter by 1. If the asset drops below the barrier before reaching D time units, the counter resets to 0; otherwise, we activate the path by marking the corresponding coordinate in the *activate* vector.

Next, we set the terminal value in the option matrix for the activated path at coordinate (N, j) to $max\{S_T - K, 0\}$ and mark *knock-in* as True for each point in the activated path. This ensures that, during computation, only paths meeting the activation conditions are considered.

STEP 4: Finally, we perform a backward induction to compute the price of the underlying asset at each node. We keep nodes where *knock-in* is False at 0 in the option matrix while using the model's formula to determine the values of other nodes in the graph.

```python
import numpy as np
import itertools

# Parameters
S0 # initial price
K # Strike price
T # Maturity (years)
N # Number of steps
r # Risk free rate
sigma # Volatility
B # Knock-in Barrier
D  # activation time
dt = T / N # rate time

#Variables initialization
u = np.exp(sigma * np.sqrt(dt))  # upward factor
d = 1 / u                        # downward factor
p = (np.exp(r * dt) - d) / (u - d)  # Risk neutral probability

S = np.zeros((N + 1, N + 1))
V = np.zeros((N + 1, N + 1))
KnockIn = np.zeros((N + 1, N + 1), dtype=bool)
S[0, 0] = S0

#STEP 1
for i in range(1, N + 1):
    for j in range(i + 1):
        S[i, j] = S0 * (u ** (i - j)) * (d ** j)

#STEP 2
chemins = list(itertools.product([0, 1], repeat=N))
activate = {chemin: False for chemin in chemins}

#STEP 3
for chemin in chemins:
    compteur = 0
    n, j = 0, 0
    positions = [(n, j)]

    for move in chemin:
        if move < N + 1:
            n += 1
            j += move
            positions.append((n, j))
            if compteur == D:
                activate[chemin] = True
            else:
                if S[n, j] >= B:
                    compteur += 1
                else:
                    compteur = 0

    if activate[chemin]:
        V[N, j] = max(S[N, j] - K, 0)
        for (i, j) in positions:
            KnockIn[i, j] = True

#STEP 4
for i in range(N - 1, -1, -1):
    for j in range(i + 1):
        if KnockIn[i, j]:
            V[i, j] = np.exp(-r * dt) * ((1 - p) * V[i+1, j] + (p) * V[i+1, j+1])

prix = V[0, 0]
print(prix)
```

Figure 3: Binomial Model Implementation in Python

In the following, we compute the price of an up-and-in Call option based on the stock price of **Total**, using the following parameters:

$$S0 = 60.75, K = 80, T = 1, N = 5, dt = T/N, r = 2.3/100, sigma = 0.69, B = 70, D = 2, dt = T/N$$

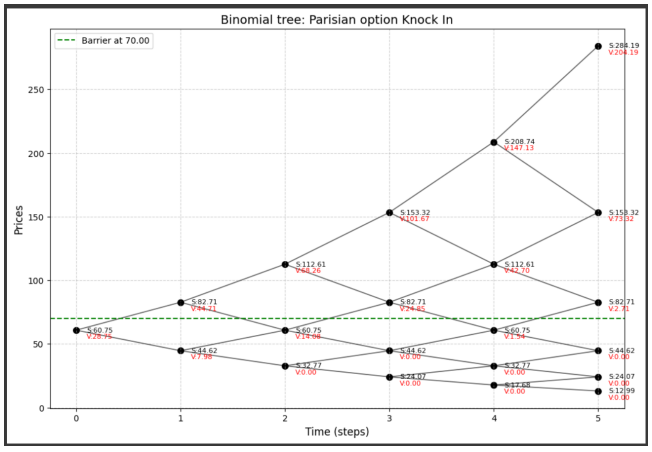


Figure 4: Parisian call up-and-in option binomial tree

And with the same script, we obtain the price of a standard Call option by setting the value of D to 0, which activates all paths.
We can also obtain the price of a down-and-in Call option by modifying the barrier condition:

```
if S[n, j] >= B:
```

with

```
if S[n, j] <= B:
```

With the price of the up-and-in Call, we obtain the price of:

- The price of the up-and-out Call by taking the difference between the price of the standard Call and the up-and-in Call. This corresponds to selling a standard Call and buying an up-and-in Call with the same characteristics.
- The price of the down-and-out Call by summing the prices of a down-and-in call option and a standard call option. This corresponds to selling a standard Call and a down-and-in Call with the same characteristics.

.

Thus, using just one script, we can obtain the price of a standard Call, up-and-in Call, up-and-out Call, down-and-in Call, and down-and-out Call.

2.2 Black–Scholes–Merton Model

2.2.1 General Framework

This model is one of the most famous in financial markets. It was introduced by Black and Scholes in 1973. It is the simplest conceivable model for describing the evolution of a risky asset while ensuring its value remains positive. This is equivalent to assuming that asset returns follow a normal distribution. Indeed, in this model, the following assumptions are made:

- The market is **frictionless** (no transaction costs, no short-selling constraints).
- There are no arbitrage opportunities.
- The market consists of a risk-free asset with value $S_0^t = e^{rt}$, and a risky asset with constant **drift** μ and **volatility** σ.
- The dynamics of the risky asset are governed by the Black–Scholes Stochastic Differential Equation (SDE), where $\sigma > 0$:
$$dS_t = \mu S_t dt + \sigma S_t dW_t$$
where W_t is a standard Brownian motion under the historical probability $\mathbb{P}$.
- The risk-free interest rate r is constant.
- The market is complete and perfect.

By applying **Itô's formula** to the discounted risky asset $X_t = e^{-rt}S_t$, we obtain the following SDE:

$$dX_t = (\mu - r)X_t\, dt + \sigma X_t\, dW_t$$

Thus, we observe that unless the drift equals the risk-free rate r, the discounted risky asset is not a martingale.
As in the discrete case, the idea of the model is to find a **risk-neutral probability** under which **any**

simple self-financing portfolio strategy, when discounted, remains a martingale. This allows us to define the value of an option as the **discounted expectation under the risk-neutral measure** of its final payoff.

This risk-neutral probability $\mathbb{Q}$ is given by **Girsanov's theorem**, which states:

$$\left.\frac{d\mathbb{Q}}{d\mathbb{P}}\right|_{\mathcal{F}_T} = Z_T = e^{-\lambda W_T - \frac{\lambda^2}{2}T}$$

where $\mathcal{F}_T$ is the natural filtration of the Brownian motion W_t up to time T.

Therefore, letting X_0 denote the option price and h_T its terminal payoff, we can write:

$$X_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}[h_T]$$

2.2.2 Barrier Options

In the case of a simple barrier option, we can express the final payoff as a function of S_T using indicator functions. In our example, we will consider, as in the case of the binomial model, a **call up in** valued at **CUI**

$$\begin{aligned} CUI &= e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[\mathbb{1}_{\{S_T>K\}\cap\{\sup S_t>L\}}\right] \\ &= e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[S_T\mathbb{1}_{\{S_T>K\}\cap\{\sup S_t>L\}}\right] \\ &\quad - Ke^{-rT}\mathbb{E}^{\mathbb{Q}}\left[\mathbb{1}_{\{S_T>K\}\cap\{\sup S_t>L\}}\right] \end{aligned}$$

We define:

$$A = \mathbb{E}^{\mathbb{Q}}\left[S_T\mathbb{1}_{\{S_T>K\}\cap\{\sup S_t>L\}}\right] \; et \; B = \mathbb{E}^{\mathbb{Q}}\left[\mathbb{1}_{\{S_T>K\}\cap\{\sup S_t>L\}}\right]$$

Lemma 1. Let (x, y) be such that $y >= 0$ and $x <= y$. Then :

$$\mathbb{Q}[X_t <= x; M_t >= y] = e^{2\mu y/\sigma^2}\mathcal{N}\left(\frac{x-2y-\mu t}{\sigma\sqrt{t}}\right)$$

where $X_t = ln(\frac{S_t}{S_0})$, $M_t = \sup(X_s, s <= t)$, $\mu = r - \frac{\sigma^2}{2}$ and $\mathcal{N}$ denotes the cumulative distribution function of a standard normal distribution.

Lemma 2. Let (x, y) be such that $y >= 0$ and $x = y$. Then :

$$\mathbb{Q}[M_t >= y] = \mathcal{N}\left(\frac{-y+\mu t}{\sigma\sqrt{t}}\right) + e^{2\mu y/\sigma^2}\mathcal{N}\left(\frac{-y-\mu t}{\sigma\sqrt{t}}\right)$$

Thus, for B we have:

$$\begin{aligned} B &= \mathbb{Q}(S_T > K, \sup(S_t) > L) \\ &= \mathbb{Q}\left(\ln\left(\frac{S_T}{S_0}\right) > \ln\left(\frac{K}{S_0}\right), \sup\ln\left(\frac{S_t}{S_0}\right) > \ln\left(\frac{L}{S_0}\right)\right) \\ &= \mathbb{Q}\left(X_T > \ln\left(\frac{K}{S_0}\right), M_T > \ln\left(\frac{L}{S_0}\right)\right) \end{aligned}$$

Using the previous lemmas, we obtain:

$$B = \mathcal{N}\left(\frac{\ln(S_0/L) + \mu T}{\sigma\sqrt{T}}\right) + \left(\frac{L}{S_0}\right)^{\frac{2r}{\sigma^2}-1}\left[\mathcal{N}\left(\frac{\ln(S_0/L) - \mu T}{\sigma\sqrt{T}}\right) - \mathcal{N}\left(\frac{\ln(KS_0/L^2) - \mu T}{\sigma\sqrt{T}}\right)\right]$$

We also have:

$$\begin{aligned} A &= \mathbb{E}^{\mathbb{Q}}\left[S_0 \exp\left(\mu T + \sigma W_T\right) \mathbb{1}_{\{S_T > K\} \cap \{\sup S_t > L\}}\right] \\ &= S_0 \exp(rT)\mathbb{E}^{\mathbb{Q}}\left[\exp\left(-\frac{\sigma^2 T}{2} + \sigma W_T\right) \mathbb{1}_{\{S_T > K\} \cap \{\sup S_t > L\}}\right] \end{aligned}$$

According to Girsanov's theorem, there exists a measure $\mathbb{Q}^*$ such that

$$\frac{d\mathbb{Q}^*}{d\mathbb{Q}} = \exp\left(\sigma W_t - \frac{\sigma^2 t}{2}\right)$$

And $\mathrm{W}_t^* = W_t - \sigma t$ is a Brownian motion under $\mathbb{Q}^*$

Then, we obtain :

$$\begin{aligned} A &= S_0 \exp(rT)\mathbb{E}^{\mathbb{Q}^*}\left[\mathbb{1}_{\{\mu T + \sigma W_T > \ln(K/S_0)\} \cap \{\sup(\mu T + \sigma W_T) > \ln(L/S_0)\}}\right] \\ &= S_0 \exp(rT)\mathbb{E}^{\mathbb{Q}^*}\left[\mathbb{1}_{\{(\mu^*)T + \sigma W_t^* > \ln(K/S_0)\} \cap \{\sup((\mu^*)T + \sigma W_t^*) > \ln(L/S_0)\}}\right] \end{aligned}$$

Thus, using the lemmas again, we obtain a result analogous to B:

$$A = S_0 e^{rT}\left[\mathcal{N}\left(\frac{\ln(S_0/L) + (\mu^*)T}{\sigma\sqrt{T}}\right) + \left(\frac{L}{S_0}\right)^{\frac{2r}{\sigma^2}+1}\left[\mathcal{N}\left(\frac{\ln(S_0/L) - (\mu^*)T}{\sigma\sqrt{T}}\right) - \mathcal{N}\left(\frac{\ln(KS_0/L^2) - (\mu^*)T}{\sigma\sqrt{T}}\right)\right]\right]$$

Therefore:

$$CUI = e^{-rT}B - Ke^{-rT}A \tag{3}$$

Thus, the Black-Scholes model allows for an exact and simple expression for the price of a standard barrier option. However, this is not the case for Parisian options.

2.2.3 Challenges in Pricing Parisian Options

In the case of Parisian options, beyond the barrier itself, one must also account for the time required to activate or deactivate the option. This prevents us from simply expressing the **final payoff h_T as a function of S_T only**, and thus complicates the exact computation of the expectation of the payoff under $\mathbb{Q}$.

As a result, we resort to an approximation.

According to the **Strong Law of Large Numbers**, we have:

$$\frac{1}{n}\sum_{i=1}^{n} h_T \xrightarrow[n\to+\infty]{\mathbb{Q}} \mathbb{E}[h_T]$$

Therefore, by simulating a large number **n** of asset paths and the corresponding terminal payoffs h_T, we can make the following approximation, where X_0 denotes the option price:

$$X_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}[h_T] \approx e^{-rT}\left(\frac{1}{n}\sum_{i=1}^{n} h_T\right)$$

This corresponds to the **standard Monte Carlo method**, which will be presented and implemented in the following sections.

2.2.4 Monte Carlo Method

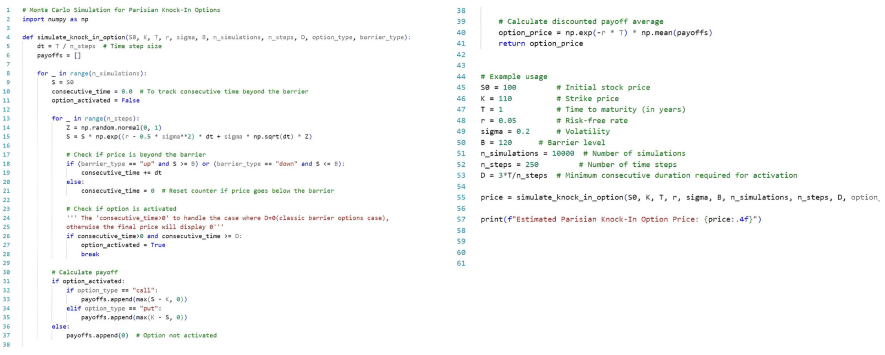
The Monte Carlo method is a numerical technique that relies on simulating numerous scenarios to estimate the value of complex financial instruments. It is particularly well-suited for barrier options, which can be of the following types:

1. **Knock-In Options:** Become active only if the underlying asset price reaches a certain level (the barrier) before expiration.
2. **Knock-Out Options:** Expire prematurely if the asset price reaches a barrier.

Principle

1. Simulate a large number of paths for the underlying asset price (e.g., via geometric Brownian motion).
2. For each path, check whether the barrier is breached (depending on the option type).
3. Compute the *payoff* for each valid path.
4. Estimate the option's value by averaging the payoffs and discounting the result to the present time.

2.2.5 Numerical implementation



```python
# Monte Carlo Simulation for Parisian Knock-In Options
import numpy as np

def simulate_knock_in_option(S0, K, T, r, sigma, B, n_simulations, n_steps, D, option_type, barrier_type):
    dt = T / n_steps  # Time step size
    payoffs = []

    for _ in range(n_simulations):
        S = S0
        consecutive_time = 0.0  # To track consecutive time beyond the barrier
        option_activated = False

        for _ in range(n_steps):
            Z = np.random.normal(0, 1)
            S = S * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z)

            # Check if price is beyond the barrier
            if (barrier_type == "up" and S >= B) or (barrier_type == "down" and S <= B):
                consecutive_time += dt
            else:
                consecutive_time = 0  # Reset counter if price goes below the barrier

            # Check if option is activated
            ''' The 'consecutive_time>0' to handle the case where D=0(classic barrier options case),
            otherwise the final price will display 0 '''
            if consecutive_time>0 and consecutive_time >= D:
                option_activated = True
                break

        # Calculate payoff
        if option_activated:
            if option_type == "call":
                payoffs.append(max(S - K, 0))
            elif option_type == "put":
                payoffs.append(max(K - S, 0))
        else:
            payoffs.append(0)  # Option not activated

    # Calculate discounted payoff average
    option_price = np.exp(-r * T) * np.mean(payoffs)
    return option_price

# Example usage
S0 = 100         # Initial stock price
K = 110          # Strike price
T = 1            # Time to maturity (in years)
r = 0.05         # Risk-free rate
sigma = 0.2      # Volatility
B = 120      # Barrier level
n_simulations = 10000  # Number of simulations
n_steps = 250        # Number of time steps
D = 3*T/n_steps  # Minimum consecutive duration required for activation

price = simulate_knock_in_option(S0, K, T, r, sigma, B, n_simulations, n_steps, D, option_

print(f"Estimated Parisian Knock-In Option Price: {price:.4f}")
```

Figure 5: Implementation of the Monte Carlo Method in Python

3 Comparison of Methods (Advantages, Limitations, Improvements...)

3.1 Binomial Model

3.1.1 Advantages

- Easy to understand: One of the main advantages of binomial trees is that they are easy to understand and implement. The method is based on simple logic and can be explained to anyone with basic knowledge of probability theory.
- Flexibility: Binomial trees are flexible and can be used to model a wide range of financial instruments, including stocks, bonds, and options. The method can also be used to analyze different types of market conditions, such as bullish or bearish trends.
- Accurate: Binomial trees are relatively accurate in predicting stock price movements, especially in the short term. The method takes into account the volatility of the underlying asset, which is a key factor in determining the probability of price changes.
- Valuable for risk management: Binomial trees are valuable tools for risk management. The method can be used to determine the probability of different outcomes and to calculate the expected returns and risks associated with different investment strategies.

3.1.2 Limitation

- Assumes Constant Volatility: One of the key assumptions of the binomial pricing model is that volatility remains constant over time. However, in reality, volatility can change rapidly and unpredictably, particularly during times of market stress. This can lead to inaccurate option valuations, as the model fails to account for the impact of sudden changes in volatility.
- Discrete Time Steps: The binomial pricing model assumes that time can be divided into a series of discrete steps. While this may be a reasonable approximation for some assets, it can be problematic for others, particularly those with irregular or unpredictable price movements. In these cases, a continuous-time model may be more appropriate.
- Limited Number of Time Steps: The accuracy of the binomial pricing model depends on the number of time steps used to model the underlying asset's price movements. However, as the number of time steps increases, the model becomes increasingly computationally intensive, which can limit its practical use.
- limited Number of price Steps: Similarly, the accuracy of the binomial pricing model also depends on the number of price steps used. However, as the number of price steps increases, the model becomes increasingly complex and difficult to interpret.
- May Fail to Capture Extreme Events: The binomial pricing model assumes that price changes are normally distributed, which means that extreme events are relatively rare. However, in reality, extreme events such as market crashes or sudden spikes in volatility can occur more frequently than the model predicts. When these events occur, the model may fail to accurately value options.

To illustrate these limitations, consider an option on a stock that experiences a sudden jump in price. The binomial pricing model may struggle to accurately value this option, as it assumes that price changes are normally distributed and volatility remains constant over time. However, in reality, the sudden jump in price may cause a corresponding jump in volatility, making it difficult to predict the stock's future movements. In this case, an alternative model such as the jump diffusion model may be more appropriate.

3.1.3 An Improvement of the Binomial Model: The Trinomial Model

Trinomial trees provide another discrete representation of stock price movement, analogous to binomial trees. The trinomial lattice has three jump parameters: u, m, and d, and three associated probabilities, denoted p_u, p_m, and p_d.

During each time step Δt, the stock price can evolve to one of three possible nodes:

- With probability p_u, it moves to the up node S_u,
- With probability p_m, it moves to the middle node S_m,
- With probability p_d, it moves to the down node S_d.

We assume that the probabilities sum to one:

$$p_m = 1 - p_u - p_d \tag{4}$$

At the end of the time step, there are five unknown parameters to determine: the probabilities p_u,p_m and p_d, and the three node values S_u, S_m, and S_d.

One approach to constructing trinomial trees is to view two steps of a binomial tree as a single step of a trinomial tree. This idea can be extended to standard binomial models with constant volatility, such as:

- CRR (Cox-Ross-Rubinstein, 1979)
- JR (Jarrow-Rudd, 1979)
- RB (Rendleman-Bartter, 1979)
- Trigeorgis (1991)
- Tian (1993, 1999)

In this work, we present the Jarrow–Rudd and Tian models:

Introduction

Trinomial trees are powerful tools for modeling the evolution of a financial asset's price in discrete time. Unlike the binomial tree where the price moves in only two directions (up or down), a trinomial tree introduces a third possibility: remaining unchanged. This allows for a better approximation of continuous-time dynamics, particularly those of the Black–Scholes model.

Two notable approaches to calibrating such trees are the models proposed by **Jarrow–Rudd (1979)** and **Tian (1993)**.

1. Jarrow–Rudd (JR) Model

The Jarrow–Rudd model constructs a symmetric trinomial tree and adjusts only the **first moment** (i.e., the mean).

Model Parameters

Movement factors:

$$u = e^{\sigma\sqrt{2\Delta t}}, \quad m = 1, \quad d = e^{-\sigma\sqrt{2\Delta t}}$$

Risk-neutral probabilities:

$$p_u = \frac{1}{6} + \frac{(r - q - \frac{\sigma^2}{2})\Delta t}{2\sigma^2}$$

$$p_m = \frac{2}{3}$$

$$p_d = 1 - p_u - p_m = \frac{1}{6} - \frac{(r - q - \frac{\sigma^2}{2})\Delta t}{2\sigma^2}$$

Calibration Objective

Match the expectation under the risk-neutral measure with:

$$\mathbb{E}^{\mathbb{Q}}[S_{t+\Delta t}] = S_t e^{(r-q)\Delta t}$$

Advantages

- Simple probability expressions
- Fast computation

Limitations

- Does not account for variance or skewness
- Less accurate for complex options

2. Tian Model (1993)

The Tian model is more advanced and calibrates the first three moments of the log-normal process: mean, variance, and skewness.

Model Parameters

Movement factors:

$$u = e^{\sigma\sqrt{2\Delta t}}, \quad m = 1, \quad d = e^{-\sigma\sqrt{2\Delta t}}$$

Risk-neutral probabilities:

$$p_u = \left(\frac{e^{r\Delta t/2} - e^{-\sigma\sqrt{\Delta t}/2}}{e^{\sigma\sqrt{\Delta t}/2} - e^{-\sigma\sqrt{\Delta t}/2}} \right)^2$$

$$p_d = \left(\frac{e^{\sigma\sqrt{\Delta t}/2} - e^{r\Delta t/2}}{e^{\sigma\sqrt{\Delta t}/2} - e^{-\sigma\sqrt{\Delta t}/2}} \right)^2$$

$$p_m = 1 - p_u - p_d$$

Calibration Objective

Match the first, second, and third moments of the discrete process with those of the log-normal Black–Scholes model.

Advantages

- Higher accuracy, particularly for exotic options (e.g., barrier, lookback, etc.)
- Faithful representation of the return distribution

Limitations

- More complex formulas
- Slightly higher computational cost

Implementation of the Tian Model

By adapting the previous code, we obtain the following implementation:

```python
# === Parametres ===
S0 # Prix initial
K     # Strike
T  # Maturité
N    # Nombre de pas
r # Taux sans risque
sigma  # Volatilité
B     # Barrière Knock-In
D   # Durée minimale au-dessus de la barrière pour activation
dt =T/N #rate time

# === Facteurs trinomial ===
u = np.exp(sigma * np.sqrt(2 * dt))  # up
d = 1 / u                            # down
m = 1                                # neutre

# === Probabilités risk-neutral ===
pu = ((np.exp(r * dt / 2) - np.exp(-sigma * np.sqrt(dt / 2))) /
      (np.exp(sigma * np.sqrt(dt / 2)) - np.exp(-sigma * np.sqrt(dt / 2)))) ** 2

pd = ((np.exp(sigma * np.sqrt(dt / 2)) - np.exp(r * dt / 2)) /
      (np.exp(sigma * np.sqrt(dt / 2)) - np.exp(-sigma * np.sqrt(dt / 2)))) ** 2

pm = 1 - pu - pd

# === Matrices ===
size = 2 * N + 1  # nombre total de niveaux de prix possibles
S = np.zeros((N + 1, size))  # Matrice des prix
V = np.zeros((N + 1, size))  # Matrice des valeurs
KnockIn = np.zeros((N + 1, size), dtype=bool)

offset = N  # pour centrer l'indice 0 au niveau initial

S[0, offset] = S0

# === Construction de l'arbre de prix ===
for i in range(1, N + 1):
    for j in range(-i, i + 1):  # j varie de -i à i
        S[i, offset + j] = S0 * (u ** j)

# === Génération des chemins (UP = 1, MIDDLE = 0, DOWN = -1) ===
mouvements = [-1, 0, 1]
chemins = list(itertools.product(mouvements, repeat=N))
activate = {chemin: False for chemin in chemins}

for chemin in chemins:
    compteur = 0
    i, j = 0, 0  # départ à la racine
    positions = [(i, j)]

    for move in chemin:
        i += 1
        j += move
        positions.append((i, j))
        prix = S[i, offset + j]

        if compteur == D:
            activate[chemin] = True
        else:
            if prix >= B:
                compteur += 1
            else:
                compteur = 0

    # Payoff final si la barrière a été activée
    if activate[chemin]:
        final_j = positions[-1][1]
        V[N, offset + final_j] = max(S[N, offset + final_j] - K, 0)
        for (i_pos, j_pos) in positions:
            KnockIn[i_pos, offset + j_pos] = True

# === Backward induction ===
for i in range(N - 1, -1, -1):
    for j in range(-i, i + 1):
        if KnockIn[i, offset + j]:
            V[i, offset + j] = np.exp(-r * dt) * (
                pu * V[i + 1, offset + j + 1] +
                pm * V[i + 1, offset + j] +
                pd * V[i + 1, offset + j - 1]
            )

# === Résultat final ===
prix_option = V[0, offset]
print("Prix de l'option parisienne knock-in (trinomial) :", prix_option)
```

Figure 6: Numerical Implementation of the Trinomial Tree for the Tian Model

With the same parameters, we obtain the following tree:

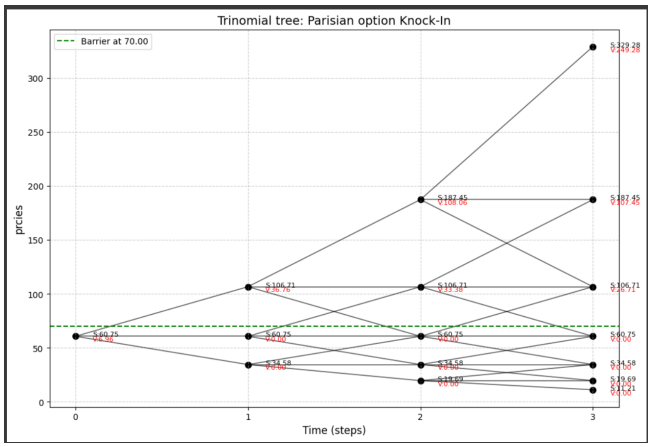


Figure 7: Parisian call up-and-in option trinomial tree

3.1.4 Comparison with the Classical Binomial Model

Advantages of the Trinomial Model

- **Greater flexibility**: The price can move up, down, or remain unchanged at each time step, which better reflects real market behavior.
- **Improved approximation of continuous models**: The trinomial model converges more rapidly to the Black–Scholes model as the number of time steps increases.
- **Enhanced numerical stability**: It reduces certain numerical errors in the case of American or exotic options.
- **Higher moment matching**: Some trinomial models, such as Tian's, allow for calibration of the first three moments (*mean, variance, skewness*).
- **Better suited for complex options**: Exhibits better performance for products like barrier options, lookback options, or Asian options.

Disadvantages of the Trinomial Model

- **Implementation complexity**: The tree structure and transition probabilities are more difficult to manage compared to the binomial case.
- **Higher computational cost**: The tree is denser with three branches per node, increasing memory and runtime requirements.
- **Less intuitive**: Less pedagogical than the binomial model for teaching basic concepts in option pricing.
- **Less universally supported**: Some software tools only implement the binomial model.

Comparative Summary

Criterion	Binomial	Trinomial
Number of directions per node	2 (up, down)	3 (up, unchanged, down)
Convergence to Black-Scholes	Moderate	Fast
Implementation complexity	Low	Moderate
Accuracy	Good	Better
Computational cost	Low	Higher
Exotic options	Less suitable	More suitable

Comparison between the binomial and trinomial models

3.2 Advantages, Limitations, and Improvement of the Monte Carlo Method

3.2.1 Advantages

This method offers several benefits when addressing financial problems. Among these advantages, we may highlight the following:

- The ability to handle complex and non-linear problems that are difficult to solve analytically or through other methods.
- It incorporates uncertainty and randomness in both inputs and outputs, while testing various scenarios and assumptions to assess their impact on the outcomes.
- It is a flexible method that can be employed to model complex financial systems that are difficult to capture with alternative techniques.
- Monte Carlo simulation is accurate and allows for the estimation of probabilities and expected outcomes based on studied data and historical assumptions.
- This method can be applied to any type of option.
- It is easily adaptable to complex dynamics of the underlying asset, such as stochastic volatility, price jumps, etc...

3.2.2 Limitations

Despite its many advantages, this method also presents several limitations when applied to option pricing. Among these, we may note:

- The method is highly dependent on the quality of the data used to generate the simulation. The results may be unreliable if the input data are inaccurate or incomplete.
- Monte Carlo simulation relies on assumptions regarding the underlying financial system. If these assumptions are incorrect, the simulation results may fail to reflect reality.

- Monte Carlo simulation requires a large number of iterations to produce accurate results. This can be computationally intensive and time-consuming.
- It is inefficient for American options and requires several adjustments in order to be applicable in that context.
- Standard Monte Carlo suffers from high variance when pricing options with rare activation conditions (e.g., deep out-of-the-money Parisian knock-ins). SMC mitigates this by concentrating sampling on more relevant trajectories.
- In Monte Carlo, many simulated paths may end up yielding zero payoff, especially in path-dependent options. SMC reallocates computational effort dynamically to the most promising particles, improving efficiency.
- Monte Carlo accuracy often deteriorates when using coarse time steps, and increasing granularity leads to higher variance. SMC maintains stable variance even with fine time discretization, allowing better approximation of continuous monitoring without variance explosion.

3.2.3 An Improvement of the Monte Carlo Method: Sequential Monte Carlo Methods

Sequential Monte Carlo (SMC) methods are used in engineering, statistics, and physics. However, their application in the field of financial option pricing remains rare. A key difference from standard Monte Carlo methods is that, in the SMC framework, simulated asset values that are rejected due to the barrier condition are resampled from asset samples that do not violate the barrier constraint, thereby improving the efficiency of the option price estimator; whereas in standard Monte Carlo, many asset paths may be discarded due to the barrier condition, making accurate option pricing more difficult.
In the following, we will compare SMC to standard Monte Carlo and show that, although SMC requires only a negligible additional effort to implement compared to the standard Monte Carlo method, it nonetheless provides a significant improvement in price estimation.
The use of SMC methods in financial mathematics is relatively recent. For example, **Del Moral** and **Patras** proposed in 2011 an SMC algorithm for computing joint default probabilities in large credit portfolios, and **Targino, Peters**, and **Shevchenko** worked in 2015 on Sequential Monte Carlo samplers for capital allocation in the context of copula risk-dependent models.

3.2.3.1 General Principle

The **Sequential Monte Carlo (SMC)** method, or particle filtering, is a refined Monte Carlo technique involving *progressive simulation* and adaptive resampling. Instead of sampling full independent paths, it evolves a set of N particles over time, each representing a potential state of the underlying asset and its associated duration counter τ.

All particles begin at $t = 0$ with $S(0)$ and $\tau = 0$. At each step $t_i \to t_{i+1}$, they are propagated via random increments drawn from the model's dynamics, updating both $S(t_{i+1})$ and $\tau(t_{i+1})$. For knock-out options, if $\tau > \theta$, a particle is "killed" by assigning it a zero (or near-zero) weight, while valid ones receive higher weights.

A resampling step follows: N particles are redrawn (with replacement) based on their weights. This duplicates promising paths and eliminates unproductive ones, with weights reset to $1/N$. This propagate–resample loop continues until maturity T.

The final option price is estimated by the average payoff of surviving particles, adjusted by the accumulated normalization factors. For knock-out options, SMC effectively samples $\mathbb{E}[P \cdot \mathbb{1}_{\text{no barrier breach during } \theta}]$ by replacing invalid trajectories with replicated valid ones, avoiding waste on paths with zero payoff.

For knock-in options, the goal is reversed: simulate prolonged barrier crossing. One can use the identity *knock-in = vanilla – knock-out* to price the knock-in component. Alternatively, SMC can directly handle knock-ins by assigning zero weight to inactive particles, and promoting those that fulfill the θ-barrier condition. Either approach allows SMC to adapt efficiently to both barrier types.

3.2.3.2 Adaptation to the θ Criterion and Barrier Crossings

The SMC method inherently accommodates continuous-time modeling via weight adjustments. Rather than using a binary condition (crossed / not crossed), each particle's weight is updated at each step by the probability that it has not yet spent θ time units beyond the barrier within $[t_i, t_{i+1}]$.

This survival probability—linked to not hitting the barrier or not staying beyond it too long—can be hard to compute in closed form, but SMC can use small time steps to approximate it. Practically, Brownian bridge techniques or analytical estimates are used to refine weights when the barrier is continuous.

Del Moral et al. suggest selecting a time step such that the probability of hitting the barrier in a single step remains small, and then adjusting particle weights by the survival probability. This integrates continuous-time effects into the weights and mitigates detection bias.

Thus, even with coarser time resolution, SMC corrects for undetected barrier hits by reweighting affected particles. Barrier crossing is treated as a probabilistic weight adjustment, not a strict elimination rule, maintaining the validity of sampled scenarios efficiently.

Figure 8 illustrates the algorithm with $M = 6$. At t_1, rejected particles (e.g., $S_1^{(4)}$, $S_1^{(6)}$) are resampled to positions $S_1^{(1)}$, $S_1^{(3)}$. Resampled particles generate new ones at the next time step. After each resampling, six particles remain active. In continuous monitoring, particles like $S_1^{(1)}, \ldots, S_1^{(5)}$ can also be rejected.

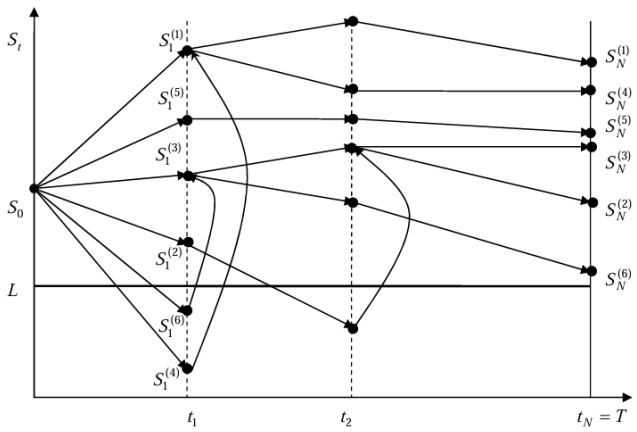


Figure 8: SMC rejection and resampling with lower barrier L

3.2.3.3 Advantages of the SMC Method

Sequential Monte Carlo (SMC) enables adaptive sampling well-suited for exotic options with rare triggers, especially Parisian barriers. It has been shown to outperform standard Monte Carlo in efficiency.

Variance reduction: SMC filters out low-payoff trajectories, focusing on relevant paths. The estimator remains unbiased, converging at $O(1/\sqrt{M})$, with much lower variance than standard MC. Its variance does not grow with the number of time steps N_t, allowing finer discretization without added noise.

Rare event efficiency: SMC maintains a higher proportion of useful particles, making it ideal for pricing deeply out-of-the-money Parisian options. It simulates rare, barrier-satisfying paths more effectively than standard MC.

No hard rejections: Instead of discarding failed paths, SMC reallocates weight from inactive to active particles, fully using all N samples and improving stability.

Flexibility: The method adapts to exotic payoffs involving multiple assets or stochastic volatility, preserving its benefits in complex settings.

3.2.3.4 Disadvantages of the SMC Method

These benefits come with challenges:

Implementation complexity: SMC is technically more involved than standard MC. It requires managing weights, choosing resampling strategies (e.g., multinomial, residual), and tracking normalization constants. Unlike MC, which averages independent payoffs, SMC must avoid introducing bias during resampling.

Limited parallelism: Resampling creates dependencies among particles, reducing parallelizability. While propagation can be parallelized, resampling is global and requires synchronization. This may limit scalability for large N, though its cost is usually minor.

Case-dependent efficiency: When the barrier condition is loose (e.g., easily breached Parisian barrier), SMC's advantage over MC shrinks. In such cases, the extra complexity may not be justified unless gains are expected for rare events or complex structures.

State dimensionality: For Parisian options, each particle must track both $S(t)$ and $\tau(t)$, doubling state dimensions. Accurately sampling the joint distribution may require more particles. If θ is large and steps are small, τ builds up gradually, risking weight collapse. Anticipatory or multiple-step resampling near $\tau \approx \theta$ can mitigate this.

In short, SMC remains highly effective for Parisian pricing by tackling variance and discretization issues directly, with only modest additional implementation effort.

3.2.3.5 Efficiency and Quantitative Comparisons

Performance differences: Both methods yield unbiased estimators with convergence rate $O(1/\sqrt{M})$, but SMC typically has much lower variance for the same M, especially when N_t is large or survival probabilities are low. Standard MC has error $\sigma/\sqrt{M}$ with σ^2 as payoff variance; when many paths yield zero payoff, this variance remains high. In contrast, SMC spreads weights across contributing particles, lowering variance. For barrier options, SMC's variance remains stable regardless of N_t, while standard MC sees increased variance with finer monitoring due to more frequent barrier hits.

Computational cost: Both methods require N_t steps over M (MC) or N (SMC) particles, so the operation count is similar. Resampling in SMC adds negligible cost ($O(N)$). However, SMC achieves similar accuracy with much smaller N: e.g., 10^5 particles vs. 10^6 MC paths. SMC also tolerates larger N_t (e.g., intraday steps) without variance explosion, whereas MC needs more simulations or variance-reduction techniques to compensate.

Accuracy and convergence: Studies show SMC often outperforms standard MC in convergence. It offers clear variance reduction (e.g., Jasra et al. 2011; Shevchenko and Del Moral 2016) with minimal complexity. While standard MC can be improved with techniques like Brownian bridges or control variates, these only partially narrow the gap. SMC's adaptive sampling closely resembles optimal importance sampling at each step—something difficult to replicate with post hoc corrections.

3.2.3.6 Concrete Use Cases

To illustrate, let us consider a few concrete scenarios involving Parisian options and the implementation of both methods:

Example 1: Reverse Convertible with a Parisian Down-and-In Barrier. In this structured product, the investor sells a down-and-in put (usually for a higher coupon). A Parisian barrier helps avoid unwanted activation from brief market drops. For instance, activation may require the underlying to close below 80% of its initial level for 5 consecutive days. This avoids triggering from a temporary 25% intraday drop.

Standard Monte Carlo: Thousands of index paths are simulated over one year, tracking time spent below 80%. Few meet the 5-day threshold, leading to many zero-payoff paths.

Sequential Monte Carlo: Particles represent ongoing trajectories. When $S < 80\%$, τ increases; otherwise, it resets. SMC concentrates on paths staying under the barrier, improving estimates of knock-in probability and expected loss. In practice, this reduces risk from transient breaches and improves pricing of the embedded put.

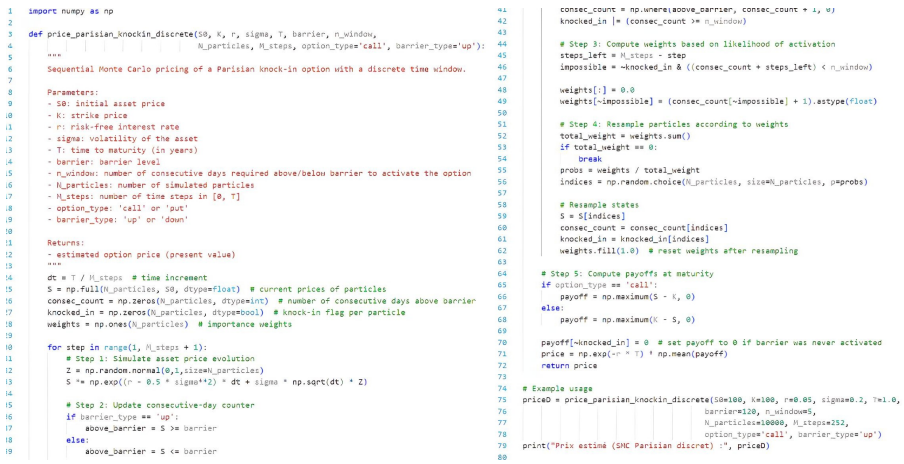
Example 2: Parisian Up-and-Out Call Option on a Volatile Stock. This option knocks out only if the price stays above 120% of its initial value for at least $\theta = 10$ consecutive days, protecting against cancellation from short-lived spikes.

Standard Monte Carlo: Simulating 252 daily steps, one checks whether the price exceeds 120% for 10 straight days. Most trajectories fall back before θ, so only a small fraction knock out. Pricing depends on the rest.

Sequential Monte Carlo: After a few days, some particles may accumulate days above 120%, others reset, and some never cross the barrier. SMC resamples to favor the latter, reducing weight on "dangerous" particles likely to knock out soon. The result is a stable estimate of the 80% surviving paths and their payoff.

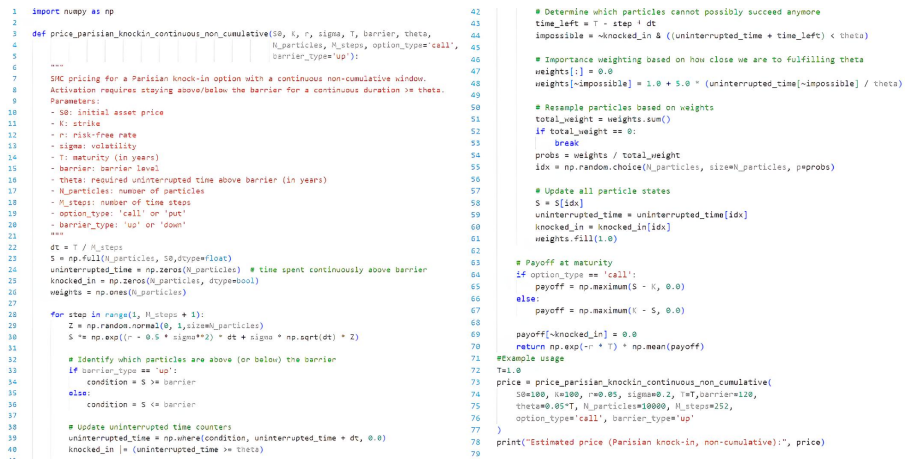
SMC not only improves pricing accuracy but also produces smoother sensitivity estimates (e.g., delta), as it captures prolonged barrier interactions that dominate the option's behavior.

3.2.3.7 Numerical Implementation



```python
import numpy as np

def price_parisian_knockin_discrete(S0, K, r, sigma, T, barrier, n_window,
                                    N_particles, M_steps, option_type='call', barrier_type='up'):
    """
    Sequential Monte Carlo pricing of a Parisian knock-in option with a discrete time window.

    Parameters:
    - S0: initial asset price
    - K: strike price
    - r: risk-free interest rate
    - sigma: volatility of the asset
    - T: time to maturity (in years)
    - barrier: barrier level
    - n_window: number of consecutive days required above/below barrier to activate the option
    - N_particles: number of simulated particles
    - M_steps: number of time steps in [0, T]
    - option_type: 'call' or 'put'
    - barrier_type: 'up' or 'down'

    Returns:
    - estimated option price (present value)
    """
    dt = T / M_steps  # time increment
    S = np.full(N_particles, S0, dtype=float)  # current prices of particles
    consec_count = np.zeros(N_particles, dtype=int)  # number of consecutive days above barrier
    knocked_in = np.zeros(N_particles, dtype=bool)  # knock-in flag per particle
    weights = np.ones(N_particles)  # importance weights

    for step in range(1, M_steps + 1):
        # Step 1: Simulate asset price evolution
        Z = np.random.normal(0,1,size=N_particles)
        S *= np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z)

        # Step 2: Update consecutive-day counter
        if barrier_type == 'up':
            above_barrier = S >= barrier
        else:
            above_barrier = S <= barrier
        consec_count = np.where(above_barrier, consec_count + 1, 0)
        knocked_in |= (consec_count >= n_window)

        # Step 3: Compute weights based on likelihood of activation
        steps_left = M_steps - step
        impossible = ~knocked_in & ((consec_count + steps_left) < n_window)

        weights[:] = 0.0
        weights[~impossible] = (consec_count[~impossible] + 1).astype(float)

        # Step 4: Resample particles according to weights
        total_weight = weights.sum()
        if total_weight == 0:
            break
        probs = weights / total_weight
        indices = np.random.choice(N_particles, size=N_particles, p=probs)

        # Resample states
        S = S[indices]
        consec_count = consec_count[indices]
        knocked_in = knocked_in[indices]
        weights.fill(1.0)  # reset weights after resampling

    # Step 5: Compute payoffs at maturity
    if option_type == 'call':
        payoff = np.maximum(S - K, 0)
    else:
        payoff = np.maximum(K - S, 0)

    payoff[~knocked_in] = 0  # set payoff to 0 if barrier was never activated
    price = np.exp(-r * T) * np.mean(payoff)
    return price

# Example usage
priceD = price_parisian_knockin_discrete(S0=100, K=100, r=0.05, sigma=0.2, T=1.0,
                                         barrier=120, n_window=5,
                                         N_particles=10000, M_steps=252,
                                         option_type='call', barrier_type='up')
print("Prix estimé (SMC Parisian discret) :", priceD)
```

Figure 9: SMC Implementation for Parisian Knock-In Option under Discrete Monitoring



```python
import numpy as np

def price_parisian_knockin_continuous_non_cumulative(S0, K, r, sigma, T, barrier, theta,
                                                     N_particles, M_steps, option_type='call',
                                                     barrier_type='up'):
    """
    SMC pricing for a Parisian knock-in option with a continuous non-cumulative window.
    Activation requires staying above/below the barrier for a continuous duration >= theta.
    Parameters:
    - S0: initial asset price
    - K: strike
    - r: risk-free rate
    - sigma: volatility
    - T: maturity (in years)
    - barrier: barrier level
    - theta: required uninterrupted time above barrier (in years)
    - N_particles: number of particles
    - M_steps: number of time steps
    - option_type: 'call' or 'put'
    - barrier_type: 'up' or 'down'
    """
    dt = T / M_steps
    S = np.full(N_particles, S0,dtype=float)
    uninterrupted_time = np.zeros(N_particles)  # time spent continuously above barrier
    knocked_in = np.zeros(N_particles, dtype=bool)
    weights = np.ones(N_particles)

    for step in range(1, M_steps + 1):
        Z = np.random.normal(0, 1,size=N_particles)
        S *= np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z)

        # Identify which particles are above (or below) the barrier
        if barrier_type == 'up':
            condition = S >= barrier
        else:
            condition = S <= barrier

        # Update uninterrupted time counters
        uninterrupted_time = np.where(condition, uninterrupted_time + dt, 0.0)
        knocked_in |= (uninterrupted_time >= theta)

        # Determine which particles cannot possibly succeed anymore
        time_left = T - step * dt
        impossible = ~knocked_in & ((uninterrupted_time + time_left) < theta)

        # Importance weighting based on how close we are to fulfilling theta
        weights[:] = 0.0
        weights[~impossible] = 1.0 + 5.0 * (uninterrupted_time[~impossible] / theta)

        # Resample particles based on weights
        total_weight = weights.sum()
        if total_weight == 0:
            break
        probs = weights / total_weight
        idx = np.random.choice(N_particles, size=N_particles, p=probs)

        # Update all particle states
        S = S[idx]
        uninterrupted_time = uninterrupted_time[idx]
        knocked_in = knocked_in[idx]
        weights.fill(1.0)

    # Payoff at maturity
    if option_type == 'call':
        payoff = np.maximum(S - K, 0.0)
    else:
        payoff = np.maximum(K - S, 0.0)

    payoff[~knocked_in] = 0.0
    return np.exp(-r * T) * np.mean(payoff)
#Example usage
T=1.0
price = price_parisian_knockin_continuous_non_cumulative(
    S0=100, K=100, r=0.05, sigma=0.2, T=T,barrier=120,
    theta=0.05*T, N_particles=10000, M_steps=252,
    option_type='call', barrier_type='up'
)
print("Estimated price (Parisian knock-in, non-cumulative):", price)
```

Figure 10: SMC Implementation for Non-Cumulative Parisian Knock-In Option under Continuous Monitoring

Key Idea of the SMC Algorithm

At each time step:

1. **Simulate the next asset value** for each particle.
2. **Track progress** toward fulfilling the Parisian condition (e.g., consecutive days or accumulated time above the barrier).
3. **Assign a weight** to each particle based on how likely it is to succeed from its current state.
4. **Resample particles** according to their weights: likely-to-succeed paths are duplicated; others are discarded.
5. Repeat until maturity.

At maturity, compute the payoff for each particle **only if the barrier condition has been fulfilled**, then average the discounted payoffs.

Mathematical Expression of the SMC Estimator

Let $h_T^{(i)}$ denote the final payoff of a valid path (i.e., one that knocked in), and let π_i denote the weight of particle i, then the SMC estimator is:

$$X_0^{\text{SMC}} \approx e^{-rT} \sum_{i=1}^{N} \pi_i h_T^{(i)}$$

3.2.3.8 Conclusions and Final Comparison Between MC and SMC

Comparative Summary

Criterion	**Standard MC**	**Sequential MC (SMC)**
Variance control	Poor	Strong
Suitability for rare events	Inefficient	Efficient
Time discretization sensitivity	High	Low
Weighting mechanism	Uniform	Adaptive
Trajectory reuse	No	Yes
Implementation complexity	Low	Moderate to High
Parallelization	Easy	More complex
Computation cost per run	Moderate	Moderate
Efficiency for exotic options	Low to moderate	High
Bias with coarse time steps	Significant	Reduced

Table 1: Comparison between Standard Monte Carlo and Sequential Monte Carlo (SMC) methods

In conclusion, both standard Monte Carlo and Sequential Monte Carlo (SMC) can price complex Parisian options, though with different approaches. Standard MC is simple but can be inefficient, while SMC adapts simulation in real time, reducing variance and better managing continuous-time constraints. For moderately sensitive options, a well-tuned standard MC may be sufficient. However, for highly sensitive cases (e.g., tight barriers or long θ), SMC offers more accurate results with moderate additional effort.

SMC excels when knock-in conditions are rare or hard to reach. By adaptively guiding simulation, it reduces variance and ensures that more paths contribute meaningfully to payoff estimation. Its adaptive nature enables it to uncover behaviors that standard MC might entirely miss, providing more reliable estimates in challenging pricing scenarios.

4 Applications of Parisian Options

After understanding how to model and value exotic products such as Parisian options, we will present some applications that Parisian options can have in various branches of finance, thanks to advanced monitoring mechanisms.

Monitoring is a generic concept that involves **examining the trajectory of an underlying asset**

over a given period and determining an action or valuation based on its behavior, interpreted here as exceeding a critical level for a specific period. The underlying assets of interest to the manager can include **a stock**, **assets**, **a debt ratio**, etc.

4.1 Case of Asset Monitoring: Banking Deposit Guarantee Funds

4.1.1 General Framework

In most countries, a deposit protection system is in place to anticipate banking failure risks, such as that of the British institution Northern Rock in 2008. This mechanism is financed by banks, the state, or a combination of both. Contributions can be paid either ex-ante (before the event occurs) or ex-post (after the event has occurred). When premiums are collected ex-ante, they are paid into the guarantee fund based on the probability of default of the participating credit institution. Therefore, it is crucial to establish pricing proportional to the risk level represented by each bank within the system, while determining the most appropriate method for evaluating these guarantees.

Two approaches are available:

- The first approach considers banking guarantees as financial derivatives and thus values them using techniques specific to such products. This analytical framework is based on the model developed by Black and Scholes. In 1977, Merton introduced a methodology to model deposit insurance under this theory.

- The second approach, actuarial by nature, consists in determining premiums based on expected average losses. These are calculated by multiplying a probability of default, typically provided by rating agencies such as *Moody's*, by the potential losses in the event of default.

Ensuring deposit security requires constant supervision of banking institutions to adjust premiums according to risk levels and reduce their tendency to take excessive risks. A Parisian-type approach involves monitoring the simultaneous evolution of assets and liabilities of financial institutions over time. The study of the Italian system will serve as an illustrative example of this mechanism.

In the Italian system, deposit guarantees are fully provided by a consortium of banks, with a possibility of reinsurance provided by the state. In case of a member's failure, this consortium covers losses up to a predefined limit, with public intervention triggered only if losses exceed this threshold. This mechanism was extensively analyzed by *De Giuli, Maggi*, and *Paris* in 2003, followed by *Bernard, Le Courtois*, and *Quittard-Pinon* in 2004.

Estimating the guarantees provided by the banking consortium and the state involves determining the appropriate premium for each stakeholder. Based on the approach developed by Merton in 1977, this evaluation is akin to calculating financial options representing the respective commitments of both parties within this co-insurance framework. Using option pricing theory, it becomes possible to quantify these guarantees numerically.

We will now examine the impact of Parisian exclusion clauses on the valuation of banking guarantees, as well as their influence on financial institutions' risk management strategies. These analyses help explain why some banks choose to increase their exposure to risk rather than reduce it. In the Italian system, consortium premiums are insufficient, and no contribution is required from the state, creating a moral hazard problem for beneficiary institutions. To address this, we propose introducing a penalty in the form of a tax paid to the consortium by banks engaging in excessive risk-taking, encouraging them to adopt more responsible management.

The operation of such a deposit protection system necessitates monitoring credit institutions. The

objective is to supervise banks to charge a premium that reflects the covered risk and to incentivize them to minimize their risk-taking. This monitoring could be of the Parisian type, meaning it is based on observing the level of assets or liabilities of the concerned credit institutions over time.

We will now illustrate these points by focusing on the **study of the Italian deposit guarantee system**.

4.1.2 Modeling the Deposit Insurance Problem Using Merton's Approach

One of the assumptions of this model is that the value of the banking guarantee corresponds to a put option on the bank's assets V. The value of this put option, at a specified maturity T, is given by:

$$max(D_T - V_T, 0) = (D_T - V_T)_+,$$

where D_T denotes the total amount of deposits at maturity. An additional assumption states that the interest rate r is constant and that the amount of guaranteed deposits remains unchanged, expressed as:

$$D_T = e^{rT} D_0,$$

which constitutes the strike price of this option.

The Italian case introduces a coverage cap provided by the consortium, denoted by K. The state then intervenes, if necessary, to cover the portion exceeding K, i.e., $D_T - V_T - K$. Consequently, the value of the guarantee provided by the state can be perceived as a put option on the assets, with a strike price of $D_T - K$ and a maturity value equal to $(D_T - V_T - K)_+$.

The guarantee provided by the consortium therefore corresponds, at maturity, to the difference between the total guarantee and that provided by the state:

$$(D_T - V_T)_+ - (D_T - V_T - K)_+.$$

This protection is thus equivalent to a *spread* between two put options. .

4.1.3 Parisian Monitoring

In the Italian co-insurance model, Merton's framework is adapted by introducing exclusion clauses designed to encourage banks to reduce their risk exposure while protecting the insurance consortium from excessively risky behavior. A bank deemed excessively risky may be expelled from the consortium. If a default occurs after this exclusion, the state then covers the entirety of the losses.

The objective of this clause is to deter financial institutions from engaging in risky behavior by confronting them with the threat of exclusion from a particularly advantageous protection system. However, we will see that this measure can lead to unintended consequences by encouraging undesirable behaviors. The implementation of this clause relies on a rigorous modeling of banks' risk-taking, which can be assessed through the observation of the evolution of their assets or debt ratio.

A bank is excluded from the consortium if its risk indicator exceeds a critical threshold for a specified period: for assets, this means remaining below a fixed level, while for leverage, it corresponds to staying above a predefined limit.

In our analysis, we assume that exclusion is based on the evolution of assets: if these remain below the critical threshold L for a continuous period of duration D, the bank is excluded. The evaluation of the guarantees provided by the consortium and the state then relies on Parisian options, whose specificity is indicated by the exponent p appearing in the notations we will introduce later.

The premiums paid to the consortium will be denoted as π_C^p, while those owed to the government will be denoted as π_G^p, when the Parisian exclusion clause is applied. Additionally, we introduce the

indicator function $\mathbb{1}^p$, which takes the value 1 if the bank meets the non-exclusion condition (i.e., its assets have not remained strictly below the critical threshold L for a duration exceeding D) and takes the value 0 in case of exclusion. This indicator $\mathbb{1}^p$ thus corresponds to the context of *down and out* Parisian options.

Conditioned on non-exclusion (and thus on our indicator $\mathbb{1}^p$), the guarantee offered by the consortium can be interpreted as a spread of *down and out* Parisian put options, whose expression is given by:

$$\pi_C^p = \mathbb{E}_{\mathbb{Q}}\left[\left((D_T - V_T)_+ - (D_T - V_T - K)_+\right)\mathbb{1}^p\right].$$

However, the total guarantee remains unchanged and is still equal to $(D_T - V_T)_+$. The only change concerns the distribution of this guarantee between the two insurers: the consortium and the state.

We observe that, *ceteris paribus*, the coverage provided by the consortium is reduced in the presence of the exclusion clause, and the portion not covered by the consortium (in case of exclusion) is borne by the state, whose value is $\pi_C - \pi_C^p$. Consequently, the guarantee provided by the government becomes:

$$\pi_G^p = \pi_G + \left(\pi_C - \pi_C^p\right).$$

We then verify that the following relation holds:

$$\pi_G^p + \pi_C^p = \pi_G + \pi_C = e^{-rT}\mathbb{E}_{\mathbb{Q}}\left[(D_T - V_T)_+\right],$$

which confirms that the sum of the guarantees provided by the consortium and the state indeed corresponds to the total value.

4.2 Monitoring a Leverage Ratio: Liquidation from the Perspective of the USA

4.2.1 General Framework

Let us now turn to one of the most original applications of Parisian options we have encountered: the modeling of the monitoring process of a company during debt renegotiation. Traditional models of corporate structure and debt valuation have generally assumed that default results in an immediate liquidation of the firm's assets.

However, certain procedures place the firm under observation, with the decision of whether it may continue operations usually made after a court-defined period.

Typically in the United States, firms benefit from the protection of **Chapter 11 of the U.S. Bankruptcy Code**, thereby avoiding the immediate liquidation of their assets.

This raises the question of whether structural models can be developed to capture default not merely as a sudden liquidation-triggering event, but also as the initiation of a **monitoring process**.

Thus, it becomes evident that legal monitoring of a firm can be approximated, at first order, by a **Parisian observation of the company's assets**.

In the following sections, we will discuss two modeling approaches: a basic Parisian framework proposed by **Pascal François and Erwan Morellec** in 2004, and a cumulative approach introduced by **Moraux in 2004**.

4.2.2 Parisian Approach

In 2004, Pascal François and Erwan Morellec established the following framework:

- The firm is financed through perpetuities. This assumption simplifies the computation of debt value by eliminating the need to account for maturity dates.

- The firm's assets follow a log-normal distribution. This ensures the continuity and predictability of asset values.
- The risk-free rate is constant, allowing the use of simple formulas to discount cash flows and providing a clear framework for valuing debt and equity.
- The model incorporates corporate taxes and bankruptcy costs on asset values. This allows the impact of taxation on firm value and financial decisions to be quantified, while also highlighting the tax shield benefit of debt, which incentivizes firms to borrow.
- Asset values are assumed to be independent of the firm's capital structure. This assumption, derived from Modigliani and Miller, facilitates analytical tractability.
- The objective is to optimize the firm's value, with particular emphasis on calculating the values of both debt and equity.

In addition to this, they incorporate a feature derived from *Chapter 11 of the U.S. Bankruptcy Code*, by assuming that default occurs when the firm's assets reach a critical level considered too low. This triggers judicial supervision, and liquidation is only declared if the assets remain continuously below this threshold for a duration exceeding the grace period granted by the court.

Their model also integrates the negotiation power shared between shareholders and bondholders in times of financial distress. Within this framework, Morellec and François derive valuation formulas for the firm, its debt, and equity, accounting for both the final bankruptcy costs in case of liquidation following failed negotiations, and the negotiation costs incurred during the distress resolution process.

These formulas involve only the Parisian time and do not depend on any Brownian motion evaluated at that time. As a result, they are numerically easy to implement, since they do not require the computation of inverse Laplace transforms.

This modeling specifically allows for the calculation of the default threshold preferred by shareholders depending on the monitoring period. Although these assumptions are useful for establishing a rigorous theoretical framework, they may not fully reflect reality for several reasons. For instance, the continuous distribution of asset values might not hold in practice, as markets can experience shocks (such as financial crises or exogenous events) that cause discontinuities in price evolution. Moreover, the assumption of a constant risk-free rate is also questionable, since in reality, interest rates fluctuate based on monetary policies, inflation, and various macroeconomic factors.

4.2.3 Cumulative Parisian Approach

Regarding Franck Moraux's 2004 approach, it involves modeling the surveillance procedure—specifically, Chapter 11 of the bankruptcy code—through a cumulative Parisian feature.

Unlike Morelle and François, Moraux points out that a firm's judicial history should not be ignored. It would be unrealistic for a company to systematically enter Chapter 11 protection to avoid paying coupons, exit before liquidation deadlines, and then repeat the process.

He emphasizes the role of judicial discretion, arguing that such procedures are not automatic and significantly affect the firm's value, equity, and debt.

Moraux proposes that the entire time spent by the firm's assets below the critical threshold should be cumulatively accounted for, whether this occurs during a single or multiple surveillance periods.

In this framework, the valuation formulas for debt and equity are expressed as **cumulative Parisian options**, which can be computed through **multiple quadrature techniques**.

In summary, unlike previous models that reset the counter after each surveillance procedure, Moraux's model considers the company's judicial history, making it arguably more realistic. However, the approach by François and Morelle might better reflect legal practices outside the U.S.

Conclusion

In this work, we studied the modeling and pricing of Parisian options by exploring different approaches, including the binomial tree method, the Black-Scholes model, and Monte Carlo simulation. The introduction of a time-monitoring mechanism makes these options more complex to analyze compared to standard barrier options.

We highlighted the advantages and limitations of each method, particularly in terms of accuracy and comPutational complexity. The choice of the most suitable method depends largely on market characteristics and investor needs.

Furthermore, beyond finance, we demonstrated that the concept of Parisian options has applications in various fields, including banking deposit guarantees and bankruptcy modeling.

As a potential avenue for further research, it would be interesting to explore more advanced models incorporating complex stochastic dynamics, as well as machine learning techniques to enhance Monte Carlo simulations.

5 Appendix

Parity Relations for Parisian Barrier Options

We denote by S the value of the underlying asset, K the strike price, L the barrier level, T the maturity, D the activation or deactivation duration, and the dividend yield δ,and r the risk-free rate.

The first useful parity relations are the following, where C and P refer to standard European call and put options:

$$\begin{aligned} &\text{(i)} \quad C = CDO + CDI = CUO + CUI \\ &\text{(ii)} \quad P = PDO + PDI = PUO + PUI \end{aligned}$$

The second parity relations are known as "mirror" (or inverse) relationships:

$$\begin{cases} PUO(S,K,L,D,T,r,\delta) &= S \cdot K \cdot CDO\left(\frac{1}{S},\frac{1}{K},\frac{1}{L},D,T,\delta,r\right) \\ PDO(S,K,L,D,T,r,\delta) &= S \cdot K \cdot CUO\left(\frac{1}{S},\frac{1}{K},\frac{1}{L},D,T,\delta,r\right) \\ PUI(S,K,L,D,T,r,\delta) &= S \cdot K \cdot CDI\left(\frac{1}{S},\frac{1}{K},\frac{1}{L},D,T,\delta,r\right) \\ PDI(S,K,L,D,T,r,\delta) &= S \cdot K \cdot CUI\left(\frac{1}{S},\frac{1}{K},\frac{1}{L},D,T,\delta,r\right) \end{cases}$$

In particular, we notice the inversion of the roles of the risk-free rate r and the dividend yield δ.
with :

$$\begin{cases} PUI & : \text{price of } \textbf{Put Up-and-In} \\ PDI & : \text{price of } \textbf{Put Down-and-In} \\ PUO & : \text{price of } \textbf{Put Up-and-Out} \\ PDO & : \text{price of } \textbf{Put Down-and-Out} \\ CUI & : \text{price of } \textbf{Call Up-and-In} \\ CDI & : \text{price of } \textbf{Call Down-and-In} \\ CUO & : \text{price of } \textbf{Call Up-and-Out} \\ CDO & : \text{price of } \textbf{Call Down-and-Out} \end{cases}$$

Proof of Lemma 1 (Page 13)

Let us define the following process:

$$X_t = \ln\left(\frac{S_t}{S_0}\right) = \left(r - \frac{\sigma^2}{2}\right)t + \sigma W_t$$

and let us denote

$$\mu = r - \frac{\sigma^2}{2}.$$

Under the risk-neutral probability $\mathbb{Q}$, we obtain:

$$dX_t = \mu\, dt + \sigma\, dW_t.$$

Let us set:

$$M_t = \sup_{0 \le s \le t} X_s.$$

We will show that for all x, y such that $y \geq 0$ and $x \leq y$, we have:

$$\mathbb{Q}(X_t \leq x\,;\, M_t \geq y) = e^{\frac{2\mu y}{\sigma^2}} \cdot \mathcal{N}\left(\frac{x - 2y - \mu t}{\sigma\sqrt{t}}\right),$$

where $\mathcal{N}(\cdot)$ denotes the cumulative distribution function of the standard normal distribution.

Let us denote by Δ (using triangular notation) the probability we aim to compute, and rewrite it as:

$$\Delta = \mathbb{Q}\left(\gamma t + W_t \leq \frac{x}{\sigma}\,;\, \sup_{s\leq t}(\gamma s + W_s) \geq \frac{y}{\sigma}\right),$$

where $\gamma = \frac{\mu}{\sigma}$.

Girsanov's theorem allows us to define a new probability measure $\mathbb{Q}^*$ such that:

$$\frac{d\widehat{\mathbb{Q}}}{d\mathbb{Q}} = L_t = \exp\left(-\gamma W_t - \frac{\gamma^2}{2}t\right),$$

and the Brownian motion under $\widehat{Q}$ is given by:

$$W_t = \widehat{W}_t - \gamma t.$$

Hence, using the indicator function of the target event, we can express:

$$\Delta = \mathbb{E}^{\mathbb{Q}}\left[\mathbf{1}_{\{\gamma t + W_t \leq \frac{x}{\sigma}\,;\, \sup_{s\leq t}(\gamma s + W_s) \geq \frac{y}{\sigma}\}}\right].$$

But for any $\mathcal{F}_t$-measurable random variable Y, it holds that:

$$\mathbb{E}^{\mathbb{Q}}[Y] = \mathbb{E}^{\widehat{\mathbb{Q}}}[L_t^{-1} Y].$$

Therefore, we obtain:

$$\Delta = \mathbb{E}^{\widehat{\mathbb{Q}}}\left[\mathbf{1}_{\{\widehat{W}_t \leq \frac{x}{\sigma}\,;\, \sup_{s\leq t}\widehat{W}_s \geq \frac{y}{\sigma}\}} \cdot \exp\left(\gamma \widehat{W}_t - \frac{\gamma^2}{2}t\right)\right].$$

Now, define the stopping time:

$$\tau(y) = \inf\left\{s \geq 0 : \widehat{W}_s \geq \frac{y}{\sigma}\right\},$$

with $\tau(y) = +\infty$ if the supremum is never reached.

We now define the reflected Brownian motion $\overline{\widehat{W}}_s$ such that:

$$\overline{\widehat{W}}_s = \begin{cases} \widehat{W}_s, & \text{if } s \in \left[0, \tau(y)^t\right] \\ \frac{2y}{\sigma} - \widehat{W}_s, & \text{if } s \in \left[\tau(y)^t, t\right], \end{cases}$$

where $\tau(y)^t = \tau(y) \wedge t$ denotes the minimum of the stopping time $\tau(y)$ and the time horizon t.

This construction corresponds to the reflection principle, applied after the process reaches the barrier level $\frac{y}{\sigma}$.

Then:

$$\Delta = \mathbb{E}^{\widehat{\mathbb{Q}}}\left[\mathbb{1}_{\left\{\overline{\widehat{W}}_t \leq \frac{x}{\sigma},\, \tau(y)\leq t\right\}} \cdot e^{\gamma\widehat{W}_t - \frac{\gamma^2 t}{2}}\right] = \mathbb{E}^{\widehat{\mathbb{Q}}}\left[\mathbb{1}_{\left\{\frac{2y}{\sigma} - \overline{\widehat{W}}_t \leq \frac{x}{\sigma},\, \tau(y)\leq t\right\}} \cdot e^{\gamma\widehat{W}_t - \frac{\gamma^2 t}{2}}\right]$$

$$\Delta = e^{\frac{2\gamma y}{\sigma}} \cdot \mathbb{E}^{\widehat{\mathbb{Q}}}\left[\mathbb{1}_{\left\{\overline{\widehat{W}}_t \geq \frac{2y-x}{\sigma}\right\}} \cdot e^{-\gamma\widehat{W}_t - \frac{\gamma^2 t}{2}}\right]$$

since $\frac{2y-x}{\sigma} \geq \frac{y}{\sigma}$, as $y \geq x$.

Using the change of variable $\overline{\widehat{W}}_t = z\sqrt{t}$ with $z \sim \mathcal{N}(0,1)$, and $\gamma = \mu/\sigma$, we obtain:

$$\begin{aligned}\Delta &= e^{\frac{2\mu y}{\sigma^2}} \cdot \int_{\frac{2y-x}{\sigma\sqrt{t}}}^{+\infty} e^{-\gamma\sqrt{t}z - \frac{\gamma^2 t}{2}} \cdot \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}\, dz \\ &= e^{\frac{2\mu y}{\sigma^2}} \cdot \int_{\frac{2y-x}{\sigma\sqrt{t}}}^{+\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{(z+\gamma\sqrt{t})^2}{2}}\, dz\end{aligned}$$

By a change of variable and using the symmetry property of Brownian motion, we obtain:

$$\Delta = e^{\frac{2\mu y}{\sigma^2}} \cdot \mathcal{N}\left(-\left(\frac{2y-x}{\sigma\sqrt{t}} + \gamma\sqrt{t}\right)\right) = e^{\frac{2\mu y}{\sigma^2}} \cdot \mathcal{N}\left(\frac{x-2y-\mu t}{\sigma\sqrt{t}}\right)$$

Proof of Lemma 2 (Page 13)

Lemma 2 is a particular case of Lemma 1.

6 Bibliography

[1] P. Poncet et R. Portait, *Finance de marché : Instruments de base, produits dérivés, portefeuille de risques*, 5e édition, Lefebvre Dalloz, 2023, pp. 508-536.

[2] M. Diener et F. Diener, *Modèles Mathématiques Discrets pour la Finance et l'Assurance*, Document LaTeX, 28 mars 2010.

[3] Carole Bernard and Olivier Le Courtois. *Le Point Sur... Les Options Parisiennes et leurs Applications*. Banque et Marchés, N°82, pp. 81-90, 2006.

[4] P. V. Shevchenko et P. Del Moral, *Valuation of Barrier Options using Sequential Monte Carlo*, preprint, disponible sur arXiv : arXiv:1405.5294v2 [q-fin.CP], 24 juillet 2015.

[5] S. Zayani, *Liens entre le modèle binomial et les équations de Black-Scholes*, Mémoire présenté comme exigence partielle de la maîtrise en mathématiques, Université du Québec à Montréal, juin 2016.

[6] P. Baldi, *Problèmes de simulation pour des options path-dependent : le rôle des grandes déviations*, Université de Roma-Tor Vergata.

[7] J. Lelong, *Étude asymptotique des algorithmes stochastiques et calcul du prix des options Parisiennes*, Thèse présentée pour obtenir le titre de Docteur de l'École Nationale des Ponts et Chaussées, Spécialité : Mathématiques appliquées, 14 septembre 2007, pp. 91-113. Jury : Monique Jeanblanc (Rapporteur), Bernard Lapeyre (Directeur de thèse), Eric Moulinès, Gilles Pages, Denis Talay, Bruno Tuffin (Examinateurs).

[8] J. C. Hull, *Options, Futures and Other Derivatives*, 5th edition, Prentice Hall

[9] Olivier Cappé, Simon J. Godsill and Eric Moulines, *An overview of existing methods and recent advances in sequential Monte Carlo*

[10] Nicolas Chopin et Omiros Papaspiliopoulos, *An Introduction to Sequential Monte Carlo*, Springer Series in Statistics ISBN 978-3-030-47844-5, pp81-129

[11] Pierre Del Moral, Arnaud Doucet et Ajay Jasra, *Sequential Monte Carlo Sampers*

[12] Carole Bernard et Phelim Boyle, *Monte Carlo Methods for Pricing Discrete Parisian Options*, 16 juin 2009

[13] fastercapital: advantages and limitations of binomial trees

[14] Le Modèle Trinomial Pour l'Évaluation des Produits Dérivés

[15] Erwan Le Saout. *Introduction aux Marchés Financiers*. 7e édition, Economica, 2022.